

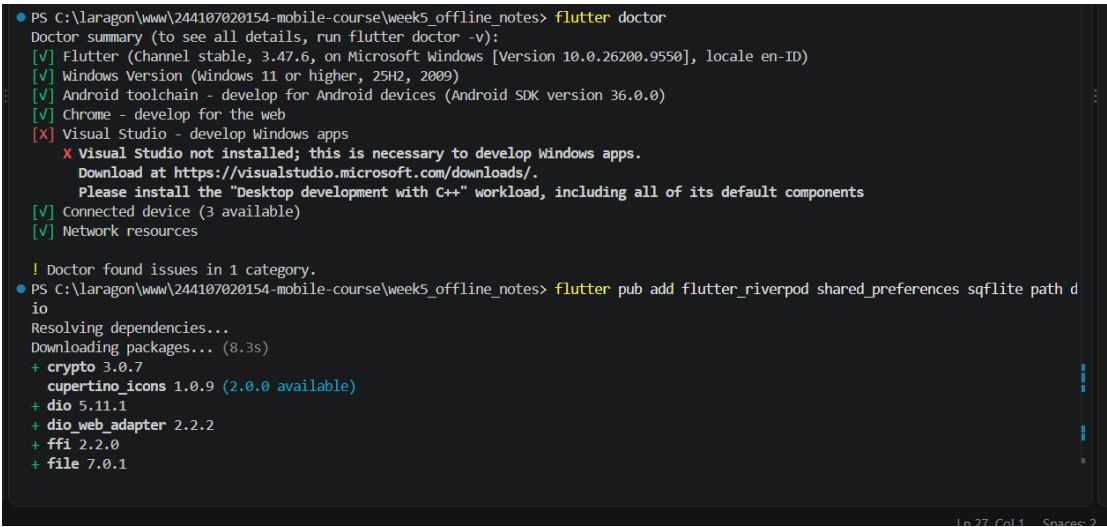


Mata Kuliah : Pemrograman Mobile
Program Studi : D4 - TEKNIK INFORMATIKA
Semester : 5

Kelas : TI – 3C
NIM : 244107020154
Nama : HAFIZ RIZQI HERNANDA
Jobsheet Ke- : 05

Laporan Jobsheet

Praktikum 1 – Preferensi dengan SharedPreferences

Langkah	Jawaban/Deskripsi
1	 Proyek menyiapkan penyimpanan preferensi untuk nilai sederhana seperti tema. SharedPreferences dipakai karena datanya berupa pasangan key-value dan tidak membutuhkan query atau relasi.
2	Buat repository preferensi lib/data/prefs.dart.

	<pre> lib > data > prefs.dart > ... 1 import 'package:shared_preferences/shared_preferences.dart'; 2 3 class PrefsRepository { 4 static const _darkModeKey = 'dark_mode'; 5 static const _lastOpenedKey = 'last_opened_at'; 6 7 Future<bool> getDarkMode() async { 8 final prefs = await SharedPreferences.getInstance(); 9 return prefs.getBool(_darkModeKey) ?? false; 10 } 11 12 Future<void> setDarkMode(bool value) async { 13 final prefs = await SharedPreferences.getInstance(); 14 await prefs.setBool(_darkModeKey, value); 15 } 16 17 Future<void> markOpenedNow() async { 18 final prefs = await SharedPreferences.getInstance(); 19 await prefs.setString(_lastOpenedKey, DateTime.now().toIso8601String()); 20 } 21 22 Future<String?> getLastOpened() async { 23 final prefs = await SharedPreferences.getInstance(); 24 return prefs.getString(_lastOpenedKey); 25 } 26 } </pre> Penjelasan: Repository memusatkan operasi baca/tulis SharedPreferences sehingga halaman UI tidak bergantung langsung pada detail penyimpanan.
3	Buat provider preferensi lib/providers/prefs_providers.dart

```

lib > providers > prefs_providers.dart > ...
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2
3  import '../data/prefs.dart';
4
5  final prefsRepositoryProvider =
6    Provider<PrefsRepository>((ref) => PrefsRepository());
7
8  final darkModeProvider =
9    AsyncNotifierProvider<DarkModeNotifier, bool>(DarkModeNotifier.new);
10
11 class DarkModeNotifier extends AsyncNotifier<bool> {
12   @override
13   Future<bool> build() => ref.watch(prefsRepositoryProvider).getDarkMode();
14
15   Future<void> toggle() async {
16     final previous = state.value ?? false;
17     final next = !previous;
18     // Optimistic update: UI langsung berubah tanpa menunggu disk.
19     state = AsyncData(next);
20     try {
21       await ref.read(prefsRepositoryProvider).setDarkMode(next);
22     } catch (_) {
23       // Gagal menyimpan: kembalikan ke nilai sebelumnya.
24       state = AsyncData(previous);
25       rethrow;
26     }
27   }
28 }
29
30 /// Membaca waktu terakhir dibuka (dari sesi sebelumnya),
31 /// lalu mencatat waktu sekarang untuk sesi berikutnya.
32 final lastOpenedProvider = FutureProvider<String?>((ref) async {
33   final repo = ref.watch(prefsRepositoryProvider);
34   final previous = await repo.getLastOpened();
35   await repo.markOpenedNow();
36   return previous;
37 });
38

```

Penjelasan: Provider membuat repository tersedia melalui Riverpod. Perubahan state preferensi dapat dibaca widget secara reaktif tanpa membuat instance penyimpanan berulang-ulang.

4

Buat halaman pengaturan lib/pages/settings_page.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3
4 import '../providers/prefs_providers.dart';
5
6 class SettingsPage extends ConsumerWidget {
7   const SettingsPage({super.key});
8
9   @override
10  Widget build(BuildContext context, WidgetRef ref) {
11    final darkAsync = ref.watch(darkModeProvider);
12    final lastOpenedAsync = ref.watch(lastOpenedProvider);
13
14    return Scaffold(
15      appBar: AppBar(title: const Text('Pengaturan')),
16      body: ListView(
17        children: [
18          darkAsync.when(
19            loading: () => const ListTile(
20              title: Text('Memuat preferensi...'),
21              trailing: SizedBox(
22                width: 24,
23                height: 24,
24                child: CircularProgressIndicator(strokeWidth: 2),
25              ), // SizedBox
26            ), // ListTile
27            error: (e, _) => ListTile(
28              leading: const Icon(Icons.error_outline),
29              title: const Text('Gagal memuat preferensi'),
30              subtitle: Text('$e'),
31            ), // ListTile
32            data: (isDark) => SwitchListTile(
33              secondary: const Icon(Icons.dark_mode_outlined),
34              title: const Text('Tema gelap'),
35              subtitle: const Text('Disimpan di SharedPreferences'),
36              value: isDark,
37              onChanged: (_) => ref.read(darkModeProvider.notifier).toggle(),
38            ), // SwitchListTile
39          ),
40          ListTile(
41            leading: const Icon(Icons.history),
42            title: const Text('Terakhir dibuka'),
43            subtitle: Text(
44              lastOpenedAsync.when(
45                data: (value) => value ?? 'Baru pertama kali dibuka',
46                loading: () => 'Memuat...',
47                error: (e, _) => 'Error: $e',
48              ),
49            ), // Text
50          ), // ListTile
51        ],
52      ), // ListView
53    ); // Scaffold
54  }
55 }

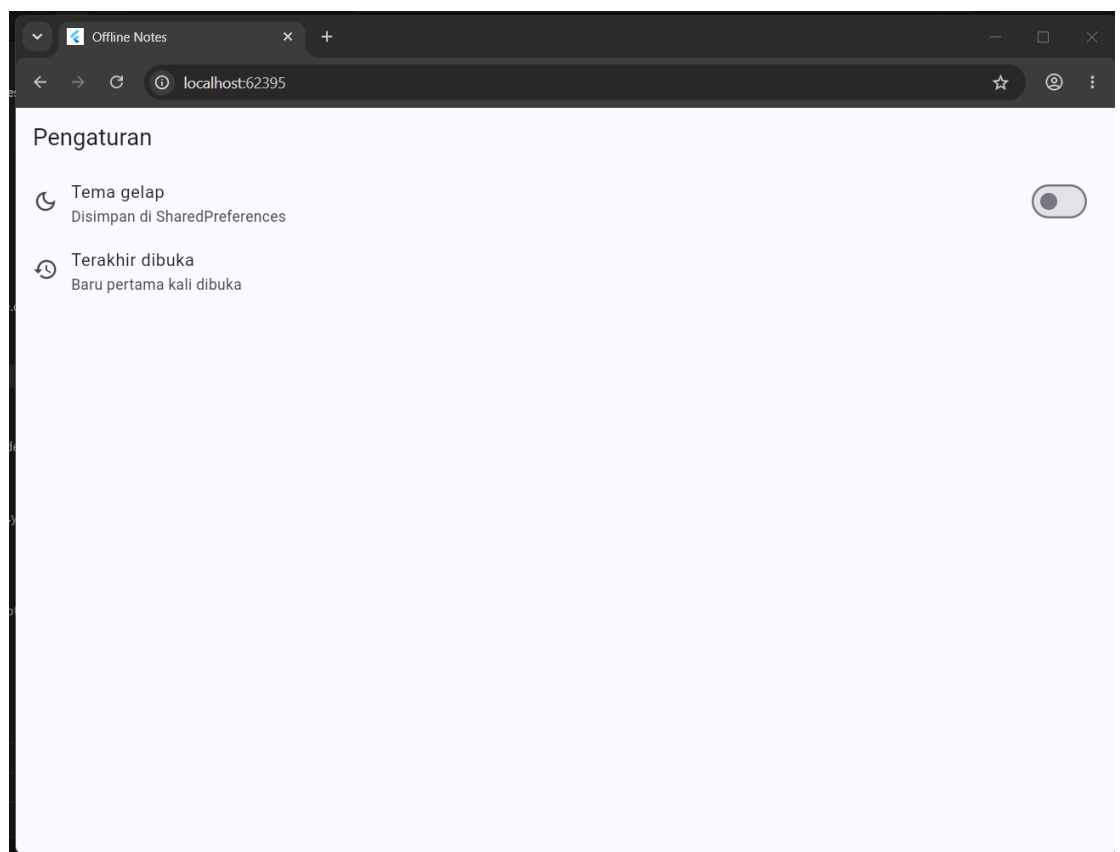
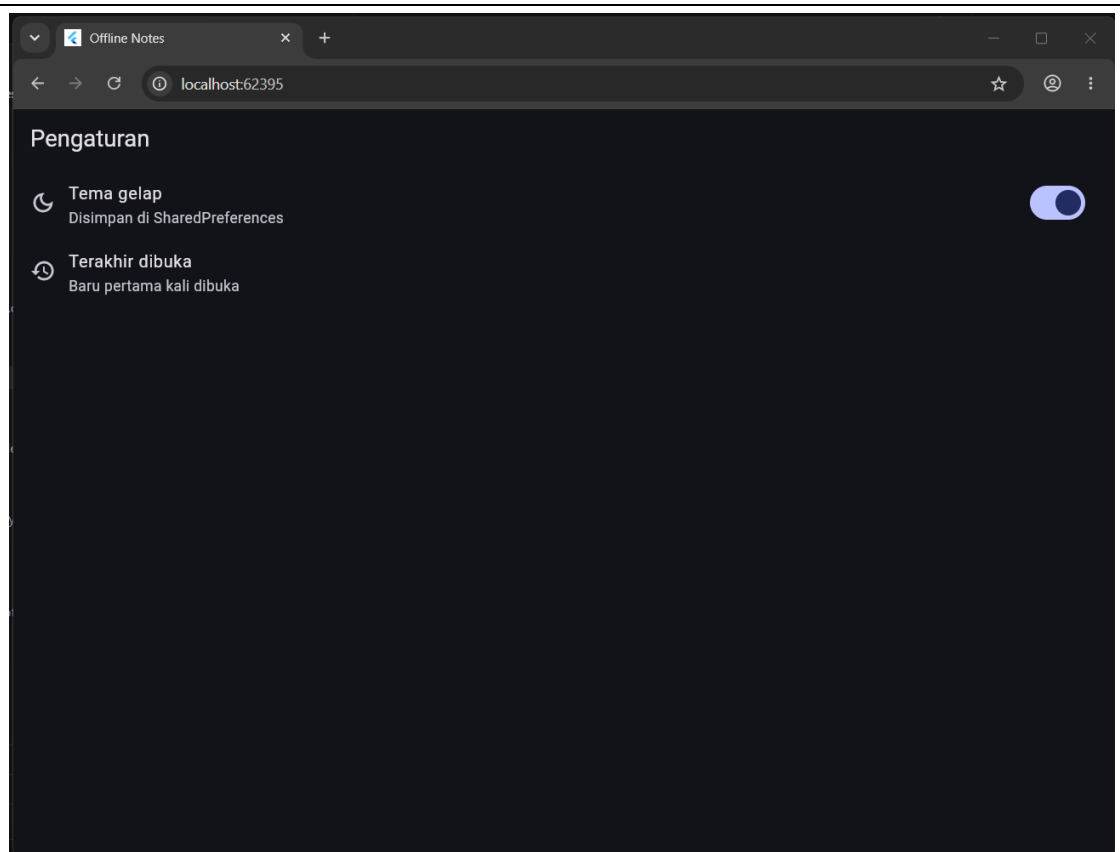
```

Penjelasan: SettingsPage menyediakan kontrol tema. Nilai switch diambil dari provider, lalu perubahan pengguna diteruskan ke aksi toggle untuk disimpan.

5

Ganti isi lib/main.dart

	<pre> 1 import 'package:flutter/material.dart'; 2 import 'package:flutter_riverpod/flutter_riverpod.dart'; 3 4 import 'pages/settings_page.dart'; 5 import 'providers/prefs_providers.dart'; 6 7 Run Debug Profile 8 void main() { 9 WidgetsFlutterBinding.ensureInitialized(); 10 runApp(const ProviderScope(child: OfflineNotesApp())); 11 } 12 13 class OfflineNotesApp extends ConsumerWidget { 14 const OfflineNotesApp({super.key}); 15 16 @override 17 Widget build(BuildContext context, WidgetRef ref) { 18 final isDark = ref.watch(darkModeProvider).value ?? false; 19 20 return MaterialApp(21 title: 'Offline Notes', 22 debugShowCheckedModeBanner: false, 23 theme: ThemeData(24 colorSchemeSeed: Colors.indigo, 25 brightness: Brightness.light, 26), // ThemeData 27 darkTheme: ThemeData(28 colorSchemeSeed: Colors.indigo, 29 brightness: Brightness.dark, 30), // ThemeData 31 themeMode: isDark ? ThemeMode.dark : ThemeMode.light, 32 home: const SettingsPage(), // diganti NotesPage pada Praktikum 3 33); // MaterialApp 34 } 35 } </pre> Penjelasan: main.dart menghubungkan ProviderScope dengan halaman utama aplikasi sehingga provider preferensi dapat dipakai oleh widget di bawahnya.
6	Hasil Run



	Penjelasan: Hasil run memperlihatkan dua keadaan tema pada halaman Pengaturan: mode gelap dan mode terang. Teks pada UI juga menunjukkan bahwa pilihan tema disimpan di SharedPreferences.
7	Pertanyaan Praktikum 1. Mengapa <code>SharedPreferences.getInstance()</code> tidak boleh dipanggil di dalam method <code>build()</code> widget? <code>SharedPreferences.getInstance()</code> bersifat asynchronous dan membuat/mengambil instance penyimpanan. Jika dipanggil di <code>build()</code>, pemanggilan dapat terjadi berulang saat widget rebuild, menambah pekerjaan async dan berisiko membuat UI tidak efisien. Karena itu instance sebaiknya diinisialisasi di layer data/provider, bukan di <code>build()</code>. 2. Jelaskan alur data dari saat switch ditekan sampai nilai tersimpan di disk dan tema berubah. Saat switch ditekan, callback memanggil <code>toggle()</code> pada provider. Provider melakukan optimistic update pada state agar UI langsung berubah, lalu nilai baru ditulis melalui repository ke SharedPreferences. Setelah penyimpanan selesai, state sudah konsisten dengan disk; bila penyimpanan gagal, error perlu ditangani agar state tidak dibiarkan berbeda dari nilai yang tersimpan. 3. Apa kelebihan dan risiko pendekatan <i>optimistic update</i> pada <code>toggle()</code>? Kelebihannya adalah UI terasa cepat karena state berubah segera tanpa menunggu I/O. Risikonya, bila proses penyimpanan gagal, terjadi ketidaksesuaian sementara antara state UI dan disk. Karena itu operasi penyimpanan sebaiknya memiliki penanganan error/revert bila diperlukan.

Praktikum 2 – Model dan Database SQLite

Langkah	Jawaban/Deskripsi
1	Buat model catatan <code>lib/data/local/note.dart</code>.

```

lib > data > local > note.dart > ...
1  class Note {
2    const Note({
3      this.id,
4      required this.title,
5      this.body = '',
6      required this.updatedAt,
7      this.dirty = false,
8    });
9
10   final int? id;
11   final String title;
12   final String body;
13   final DateTime updatedAt;
14   final bool dirty;
15
16   Note copyWith({
17     int? id,
18     String? title,
19     String? body,
20     DateTime? updatedAt,
21     bool? dirty,
22   }) {
23     return Note(
24       id: id ?? this.id,
25       title: title ?? this.title,
26       body: body ?? this.body,
27       updatedAt: updatedAt ?? this.updatedAt,
28       dirty: dirty ?? this.dirty,
29     );
30   }
31
32   Map<String, Object?> toMap() => {
33     'id': id,
34     'title': title,
35     'body': body,
36     'updated_at': updatedAt.toIso8601String(),
37     'dirty': dirty ? 1 : 0,
38   };
39
40   factory Note.fromMap(Map<String, Object?> map) {
41     return Note(
42       id: (map['id'] as num?)?.toInt(),
43       title: map['title'] as String? ?? '',
44       body: map['body'] as String? ?? '',
45       updatedAt: DateTime.tryParse(map['updated_at'] as String? ?? '') ??
46         DateTime.fromMillisecondsSinceEpoch(0),
47       dirty: ((map['dirty'] as num?)?.toInt() ?? 0) == 1,
48     ); // Note
49   }
50 }

```

Penjelasan: Model Note mewakili data catatan yang digunakan aplikasi, termasuk identitas catatan, isi, waktu perubahan, dan penanda dirty untuk kebutuhan sinkronisasi.

2

Buat pembuka database lib/data/local/db.dart.

```

lib > data > local > db.dart > ...
1  import 'package:path/path.dart' as p;
2  import 'package:sqflite/sqflite.dart';
3
4  Future<Database> openNotesDb() async {
5    final dir = await getDatabasesPath();
6    return openDatabase(
7      p.join(dir, 'offline_notes.db'),
8      version: 1,
9      onCreate: (db, version) async {
10       await db.execute('''
11         CREATE TABLE notes(
12           id INTEGER PRIMARY KEY AUTOINCREMENT,
13           title TEXT NOT NULL,
14           body TEXT NOT NULL DEFAULT '',
15           updated_at TEXT NOT NULL,
16           dirty INTEGER NOT NULL DEFAULT 0
17         )
18       ''');
19       await db.execute('''
20         CREATE TABLE cached_posts(
21           id INTEGER PRIMARY KEY,
22           payload TEXT NOT NULL,
23           cached_at TEXT NOT NULL
24         )
25       ''');
26     },
27   );
28 }

```

Penjelasan: Pembuka database membuat atau membuka database SQLite lokal dan memastikan struktur tabel tersedia sebelum repository melakukan operasi CRUD.

3

Buat repository lib/data/repositories/note_repository.dart.

```

lib > data > repositories > note_repository.dart > ...
1  import 'package:sqflite/sqflite.dart';
2
3  import '../local/db.dart';
4  import '../local/note.dart';
5
6  class NoteRepository {
7      NoteRepository({Future<Database> Function()? openDb})
8          : _openDb = openDb ?? openNotesDb;
9
10     final Future<Database> Function() _openDb;
11
12     Future<List<Note>> fetchNotes() async {
13         final db = await _openDb();
14         final rows = await db.query('notes', orderBy: 'updated_at DESC');
15         return rows.map(Note.fromMap).toList();
16     }
17
18     Future<Note?> getNoteById(int id) async {
19         final db = await _openDb();
20         final rows = await db.query(
21             'notes',
22             where: 'id = ?',
23             whereArgs: [id],
24             limit: 1,
25         );
26         return rows.isEmpty ? null : Note.fromMap(rows.first);
27     }
28
29     Future<Note> addNote({required String title, String body = ''}) async {
30         final db = await _openDb();
31         final note = Note(
32             title: title,
33             body: body,
34             updatedAt: DateTime.now(),
35             dirty: true,
36         ); // Note
37         final id = await db.insert('notes', note.toMap());
38         return note.copyWith(id: id);
39     }

```

```

40
41 Future<void> updateNote(Note note) async {
42     if (note.id == null) {
43         throw ArgumentError('Catatan belum memiliki id');
44     }
45     final db = await _openDb();
46     final updated = note.copyWith(updatedAt: DateTime.now(), dirty: true);
47     await db.update(
48         'notes',
49         updated.toMap(),
50         where: 'id = ?',
51         whereArgs: [note.id],
52     );
53 }
54
55 Future<void> deleteNote(int id) async {
56     final db = await _openDb();
57     await db.delete('notes', where: 'id = ?', whereArgs: [id]);
58 }
59
60 Future<int> countDirty() async {
61     final db = await _openDb();
62     final rows = await db.rawQuery(
63         'SELECT COUNT(*) AS c FROM notes WHERE dirty = 1',
64     );
65     return (rows.first['c'] as num?).toInt() ?? 0;
66 }
67
68 Future<void> markAllSynced() async {
69     final db = await _openDb();
70     await db.update('notes', {'dirty': 0}, where: 'dirty = 1');
71 }
72 }

```

Penjelasan: Repository menjadi lapisan akses data: UI/provider memanggil metode repository, sedangkan detail SQL, transaksi, dan konversi baris database ke objek Note ditangani di satu tempat.

4

Pertanyaan Praktikum

1. Mengapa kolom dirty bertipe INTEGER dan bukan BOOLEAN?

SQLite tidak memiliki tipe BOOLEAN native seperti di beberapa database lain. Karena itu nilai boolean biasa direpresentasikan sebagai INTEGER, umumnya 0 untuk false dan 1 untuk true; pendekatan ini juga konsisten dengan tipe SQLite.

2. Apa fungsi parameter openDb pada constructor NoteRepository?

openDb memungkinkan NoteRepository menerima fungsi pembuka database dari luar. Ini memudahkan dependency injection, pengujian dengan

	database/fake lain, dan membuat repository tidak terkunci pada satu cara membuka database. 3. Mengapa query memakai where: 'id = ?' dan whereArgs, bukan interpolasi string? where: 'id = ?' dan whereArgs memisahkan SQL dari nilai parameter. Cara ini lebih aman daripada interpolasi string karena membantu mencegah SQL injection dan menghindari masalah escaping/format nilai. 4. Apa yang terjadi jika Anda menambah kolom baru di onCreate tanpa menaikkan version? onCreate hanya dijalankan ketika database pertama kali dibuat. Jika database pengguna sudah ada dan version tidak dinaikkan, perubahan kolom di onCreate tidak otomatis dijalankan. Akibatnya database lama tetap memakai skema lama dan query yang membutuhkan kolom baru dapat gagal.
--	---

Praktikum 3 - Halaman Catatan Offline dengan Riverpod

Langkah	Jawaban/Deskripsi
1	Buat provider catatan lib/providers/note_providers.dart.

```

lib > providers > note_providers.dart > ...
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2
3  import '../data/local/note.dart';
4  import '../data/repositories/note_repository.dart';
5
6  final noteRepositoryProvider =
7    Provider<NoteRepository>((ref) => NoteRepository());
8
9  final notesProvider = FutureProvider<List<Note>>{
10    (ref) => ref.watch(noteRepositoryProvider).fetchNotes(),
11  };
12
13  final dirtyCountProvider = FutureProvider<int>{
14    (ref) => ref.watch(noteRepositoryProvider).countDirty(),
15  };
16
17  final noteActionsProvider = Provider<NoteActions>((ref) => NoteActions(ref));
18
19  /// Kumpulan aksi yang mengubah data. Setiap mutasi diakhiri invalidate
20  /// agar provider baca (notes & dirty count) memuat ulang dari database.
21  class NoteActions {
22    NoteActions(this._ref);
23
24    final Ref _ref;
25
26    NoteRepository get _repo => _ref.read(noteRepositoryProvider);
27
28    Future<void> add(String title, String body) async {
29      await _repo.addNote(title: title, body: body);
30      _refresh();
31    }
32
33    Future<void> update(Note note) async {
34      await _repo.updateNote(note);
35      _refresh();
36    }
37
38    Future<void> delete(int id) async {
39      await _repo.deleteNote(id);
40      _refresh();
41    }
42
43    void _refresh() {
44      _ref.invalidate(notesProvider);
45      _ref.invalidate(dirtyCountProvider);
46    }
47  }

```

Penjelasan: Provider catatan mengelola state dan aksi CRUD melalui Riverpod sehingga perubahan data dapat diteruskan ke widget tanpa membuat akses database di dalam build().

2

Buat dialog form catatan lib/widgets/note_form_dialog.dart.

```

lib > widgets > note_form_dialog.dart > ...
1  import 'package:flutter/material.dart';
2
3  import '../data/local/note.dart';
4
5  typedef NoteFormResult = ({String title, String body});
6
7  class NoteFormDialog extends StatefulWidget {
8    const NoteFormDialog({super.key, this.initial});
9
10   final Note? initial;
11
12   @override
13   State<NoteFormDialog> createState() => _NoteFormDialogState();
14 }
15
16 class _NoteFormDialogState extends State<NoteFormDialog> {
17   late final TextEditingController _title =
18     TextEditingController(text: widget.initial?.title ?? '');
19   late final TextEditingController _body =
20     TextEditingController(text: widget.initial?.body ?? '');
21   String? _error;
22
23   @override
24   void dispose() {
25     _title.dispose();
26     _body.dispose();
27     super.dispose();
28   }
29
30   void _submit() {
31     final title = _title.text.trim();
32     if (title.isEmpty) {
33       setState(() => _error = 'Judul wajib diisi');
34       return;
35     }
36     Navigator.of(context).pop<NoteFormResult>(
37       (title: title, body: _body.text.trim()),
38     );
39   }

```

```

40
41 @override
42 Widget build(BuildContext context) {
43   return AlertDialog(
44     title: Text(widget.initial == null ? 'Catatan baru' : 'Ubah catatan'),
45     content: Column(
46       mainAxisAlignment: MainAxisAlignment.min,
47       children: [
48         TextField(
49           controller: _title,
50           autofocus: true,
51           decoration: InputDecoration(labelText: 'Judul', errorText: _error),
52         ), // TextField
53         const SizedBox(height: 8),
54         TextField(
55           controller: _body,
56           maxLines: 4,
57           decoration: const InputDecoration(labelText: 'Isi catatan'),
58         ), // TextField
59       ],
60     ), // Column
61     actions: [
62       TextButton(
63         onPressed: () => Navigator.of(context).pop(),
64         child: const Text('Batal'),
65       ), // TextButton
66       FilledButton(onPressed: _submit, child: const Text('Simpan')),
67     ],
68   ); // AlertDialog
69 }
70 }

```

Penjelasan: Dialog form memisahkan input judul dan isi dari halaman utama. Validasi mencegah penyimpanan ketika judul masih kosong.

3

Buat halaman catatan lib/pages/notes_page.dart.

```

lib > pages > notes_page.dart > ...
1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3
4  import '../data/local/note.dart';
5  import '../providers/note_providers.dart';
6  import '../widgets/note_form_dialog.dart';
7  import 'settings_page.dart';
8
9  class NotesPage extends ConsumerWidget {
10   const NotesPage({super.key});
11
12   Future<void> _openForm(
13     BuildContext context,
14     WidgetRef ref, [
15       Note? note,
16   ]) async {
17     final result = await showDialog<NoteFormResult>(
18       context: context,
19       builder: (_) => NoteFormDialog(initial: note),
20     );
21     if (result == null) return; // dibatalkan
22     final actions = ref.read(noteActionsProvider);
23     if (note == null) {
24       await actions.add(result.title, result.body);
25     } else {
26       await actions.update(
27         note.copyWith(title: result.title, body: result.body),
28       );
29     }
30   }

```

```

32  @override
33  Widget build(BuildContext context, WidgetRef ref) {
34    final notesAsync = ref.watch(notesProvider);
35    final dirty = ref.watch(dirtyCountProvider).value ?? 0;
36
37    return Scaffold(
38      appBar: AppBar(
39        title: const Text('Offline Notes'),
40        actions: [
41          Tooltip(
42            message: 'Catatan belum tersinkron',
43            child: Badge(
44              isLabelVisible: dirty > 0,
45              label: Text('$dirty'),
46              child: const Icon(Icons.cloud_upload_outlined),
47            ),
48          ),
49          IconButton(
50            tooltip: 'Pengaturan',
51            icon: const Icon(Icons.settings_outlined),
52            onPressed: () => Navigator.of(context).push(
53              MaterialPageRoute(builder: (_) => const SettingsPage()),
54            ),
55          ),
56        ],
57      ),
58      body: notesAsync.when(
59        loading: () => const Center(child: CircularProgressIndicator()),
60        error: (e, _) => _ErrorView(
61          message: 'Gagal membaca database: $e',
62          onRetry: () => ref.invalidate(notesProvider),
63        ),
64        data: (notes) {
65          if (notes.isEmpty) return const _EmptyView();
66          return ListView.separated(
67            itemCount: notes.length,
68            separatorBuilder: (context, index) => const Divider(height: 1),
69            itemBuilder: (context, index) {
70              final note = notes[index];
71              return ListTile(
72                leading: Icon(
73                  note.dirty ? Icons.cloud_off : Icons.cloud_done,
74                  color: note.dirty ? Colors.orange : Colors.green,
75                ),
76                title: Text(note.title),
77                subtitle: Text(
78                  note.body.isEmpty ? '(tanpa isi)' : note.body,
79                  maxLines: 1,
80                  overflow: TextOverflow.ellipsis,
81                ),
82                onTap: () => _openForm(context, ref, note),
83                trailing: IconButton(
84                  tooltip: 'Hapus',
85                  icon: const Icon(Icons.delete_outline),
86                  onPressed: () =>
87                    ref.read(noteActionsProvider).delete(note.id!),

```

```

88     ),
89     );
90   },
91   );
92   },
93   ),
94   floatingActionButton: FloatingActionButton.extended(
95     onPressed: () => _openForm(context, ref),
96     icon: const Icon(Icons.add),
97     label: const Text('Catatan'),
98   ),
99   );
100 }
101 }
102
103 class _EmptyView extends StatelessWidget {
104   const _EmptyView();
105
106   @override
107   Widget build(BuildContext context) {
108     return const Center(
109       child: Column(
110         mainAxisAlignment: MainAxisAlignment.min,
111         children: [
112           Icon(Icons.note_alt_outlined, size: 64),
113           SizedBox(height: 12),
114           Text('Belum ada catatan. Tekan + untuk menambah.'),
115         ],
116       ),
117     );
118   }
119 }
120
121 class _ErrorView extends StatelessWidget {
122   const _ErrorView({required this.message, required this.onRetry});
123
124   final String message;
125   final VoidCallback onRetry;
126
127   @override
128   Widget build(BuildContext context) {
129     return Center(
130       child: Padding(
131         padding: const EdgeInsets.all(24),
132         child: Column(
133           mainAxisAlignment: MainAxisAlignment.min,
134           children: [
135             const Icon(Icons.error_outline, size: 48),
136             const SizedBox(height: 12),
137             Text(message, textAlign: TextAlign.center),
138             const SizedBox(height: 12),
139             FilledButton(onPressed: onRetry, child: const Text('Coba lagi')),
140           ],
141         ),
142       ),
143     );

```

Penjelasan: NotesPage menampilkan daftar catatan, indikator dirty, tombol aksi, serta state loading/error. Data berasal dari provider sehingga halaman ikut diperbarui setelah mutasi.

4

Jadikan NotesPage sebagai halaman utama di lib/main.dart.

```

lib > main.dart > OfflineNotesApp > build
1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3
4  import 'pages/notes_page.dart';
5  import 'providers/prefs_providers.dart';
6
7  void main() {
8    WidgetsFlutterBinding.ensureInitialized();
9    runApp(const ProviderScope(child: OfflineNotesApp()));
10 }
11
12 class OfflineNotesApp extends ConsumerWidget {
13   const OfflineNotesApp({super.key});
14
15   @override
16   Widget build(BuildContext context, WidgetRef ref) {
17     final isDark = ref.watch(darkModeProvider).value ?? false;
18
19     return MaterialApp(
20       title: 'Offline Notes',
21       debugShowCheckedModeBanner: false,
22       theme: ThemeData(
23         colorSchemeSeed: Colors.indigo,
24         brightness: Brightness.light,
25       ), // ThemeData
26       darkTheme: ThemeData(
27         colorSchemeSeed: Colors.indigo,
28         brightness: Brightness.dark,
29       ), // ThemeData
30       themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
31       home: const NotesPage(), // diganti NotesPage pada Praktikum 3
32     ); // MaterialApp
33   }
34 }
35

```

Penjelasan: NotesPage dijadikan halaman utama sehingga saat aplikasi dibuka pengguna langsung melihat daftar catatan offline.

5

Uji CRUD dan mode pesawat.





	Penjelasan: Pengujian difokuskan pada alur CRUD lokal dan kondisi mode pesawat. Penyimpanan SQLite tetap dapat dibaca/tulis tanpa koneksi jaringan.
6	Pertanyaan Praktikum 1. Mengapa setelah setiap mutasi perlu meng-invalidate notesProvider dan dirtyCountProvider? Apa yang terjadi jika hanya salah satu? notesProvider perlu di-invalidate agar daftar catatan membaca data terbaru. dirtyCountProvider juga perlu di-invalidate agar jumlah catatan yang belum tersinkron ikut diperbarui. Jika hanya notesProvider yang di-invalidate, daftar dapat benar tetapi badge/jumlah dirty masih lama; jika hanya dirtyCountProvider, jumlah dapat benar tetapi daftar masih menampilkan data lama. 2. Bagaimana cara Anda memicu state error secara sengaja untuk menguji tampilan _ErrorView? State error dapat dipicu secara sengaja dengan membuat FakeNoteRepository mengembalikan/throw Exception saat fetchNotes(), atau meng-override provider pada pengujian agar melempar error. Setelah provider dibaca, UI akan masuk state error dan _ErrorView dapat diverifikasi. 3. Mengapa aplikasi tetap berfungsi dalam mode pesawat walaupun tidak ada kode khusus untuk mode offline? Aplikasi tetap berfungsi karena operasi CRUD catatan dilakukan pada database SQLite lokal. Mode pesawat hanya memutus jaringan; selama fitur yang diuji tidak membutuhkan request network, data lokal masih dapat dibaca dan ditulis.

Tabel Uji Praktikum 3

Skenario Uji	Hasil Aktual	Status (Lulus/Gagal)
Validasi tambah catatan tanpa judul	Form menampilkan pesan "Judul wajib diisi" sehingga data kosong tidak disimpan.	Lulus
Tampilan catatan lokal	Catatan "Hari ini" terlihat pada daftar sehingga data lokal dapat dibaca kembali.	Lulus

Mode pesawat/offline	Screenshot menunjukkan indikator mode pesawat, sementara aplikasi tetap menampilkan data lokal.	Lulus
----------------------	---	-------

Praktikum 4 – Offline First : Cache-First dan Sinkronisasi

Langkah	Jawaban/Deskripsi
1	Buat saklar offline lib/providers/offline_providers.dart. <pre>lib > providers > offline_providers.dart > ... 1 import 'package:flutter_riverpod/flutter_riverpod.dart'; 2 3 /// Simulasi offline yang deterministik untuk demo dan testing, 4 /// sehingga tidak bergantung pada kondisi Wi-Fi kelas. 5 final forceOfflineProvider = 6 NotifierProvider<ForceOfflineNotifier, bool>(ForceOfflineNotifier.new); 7 8 class ForceOfflineNotifier extends Notifier<bool> { 9 @override 10 bool build() => false; 11 12 void toggle() => state = !state; 13 } </pre> Penjelasan: forceOffline menjadi saklar simulasi offline yang deterministik. Dengan cara ini perilaku tanpa jaringan dapat diuji tanpa bergantung pada kondisi Wi-Fi atau emulator.
2	Buat logika sinkronisasi lib/data/sync.dart.

```

lib > data > sync.dart > ...
1 import 'local/note.dart';
2 import 'repositories/note_repository.dart';
3
4 class OfflineException implements Exception {
5   const OfflineException(this.message);
6
7   final String message;
8
9   @override
10  String toString() => 'OfflineException: $message';
11 }
12
13 // Mengirim catatan dirty ke "server" (simulasi) lalu menandainya bersih.
14 // Mengembalikan jumlah catatan yang tersinkron.
15 Future<int> syncNotes(
16   NoteRepository repo, {
17     bool offline = false,
18     Duration latency = const Duration(seconds: 1),
19   }) async {
20   if (offline) {
21     throw const OfflineException('Perangkat offline, sinkronisasi ditunda.');

```

Penjelasan: syncNotes mengambil catatan dirty lalu mencoba mengirim perubahan. Hanya item yang berhasil diproses yang semestinya ditandai sudah tersinkron.

3

Tambahkan aksi sync() ke kelas NoteActions di note_providers.dart.

```

lib > providers > note_providers.dart > NoteActions
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2
3  import '../data/local/note.dart';
4  import '../data/repositories/note_repository.dart';
5  import '../data/sync.dart';
6  import 'offline_providers.dart';
7
8
9  final noteRepositoryProvider =
10 |   Provider<NoteRepository>((ref) => NoteRepository());
11
12  final notesProvider = FutureProvider<List<Note>>()
13 |   (ref) => ref.watch(noteRepositoryProvider).fetchNotes(),
14 | );
15
16  final dirtyCountProvider = FutureProvider<int>()
17 |   (ref) => ref.watch(noteRepositoryProvider).countDirty(),
18 | );
19
20  final noteActionsProvider = Provider<NoteActions>((ref) => NoteActions(ref));
21
22  /// Kumpulan aksi yang mengubah data. Setiap mutasi diakhiri invalidate
23  /// agar provider baca (notes & dirty count) memuat ulang dari database.
24  class NoteActions {
25 |   NoteActions(this._ref);
26
27 |   final Ref _ref;
28
29 |   NoteRepository get _repo => _ref.read(noteRepositoryProvider);
30
31 |   Future<int> sync() async {
32 |     final offline = _ref.read(forceOfflineProvider);
33 |     final count = await syncNotes(_repo, offline: offline);
34 |     _refresh();
35 |     return count;
36 |   }
37

```

Penjelasan: Aksi sync ditempatkan pada NoteActions agar UI cukup memanggil satu provider action, kemudian daftar catatan dan jumlah dirty dapat diperbarui kembali.

4

Buat model lib/data/remote/post.dart.

```

lib > data > remote > post.dart > ...
1  class Post {
2 |   const Post({required this.id, required this.title, required this.body});
3
4 |   final int id;
5 |   final String title;
6 |   final String body;
7
8 |   factory Post.fromJson(Map<String, dynamic> json) => Post(
9 |     id: (json['id'] as num).toInt(),
10 |     title: json['title'] as String? ?? '',
11 |     body: json['body'] as String? ?? '',
12 |   );
13
14 |   Map<String, dynamic> toJson() => {'id': id, 'title': title, 'body': body};
15 }

```

	Penjelasan: Model Post memetakan data dari sumber remote agar struktur respons tidak bercampur dengan model Note lokal.
5	Buat repository posts dengan cache lib/data/repositories/post_repository.dart. <pre> lib > data > repositories > post_repository.dart > PostRepository > PostRepository 1 import 'dart:convert'; 2 3 import 'package:dio/dio.dart'; 4 import 'package:sqflite/sqflite.dart'; 5 6 import '../local/db.dart'; 7 import '../remote/post.dart'; 8 9 class PostRepository { 10 PostRepository({Dio? dio, Future<Database> Function()? openDb}) 11 : _dio = dio ?? 12 Dio(BaseOptions(13 baseUrl: 'https://jsonplaceholder.typicode.com', 14 connectTimeout: const Duration(seconds: 5), 15 receiveTimeout: const Duration(seconds: 5), 16)), // BaseOptions // Dio 17 _openDb = openDb ?? openNotesDb; 18 19 final Dio _dio; 20 final Future<Database> Function() _openDb; 21 22 /// Baca cache dari tabel cached_posts (tanpa jaringan). 23 Future<List<Post>> readCachedPosts() async { 24 final db = await _openDb(); 25 final rows = await db.query('cached_posts', orderBy: 'id ASC'); 26 return rows 27 .map((r) => Post.fromJson(28 jsonDecode(r['payload'] as String) as Map<String, dynamic>, 29)) 30 .toList(); 31 } 32 33 /// Ambil dari jaringan, lalu ganti isi cache dalam satu transaksi. 34 Future<List<Post>> fetchAndCache() async { 35 final res = await _dio.get<List<dynamic>>('/posts'); 36 final posts = (res.data ?? const []) 37 .map((e) => Post.fromJson(e as Map<String, dynamic>)) 38 .toList(); 39 40 final db = await _openDb(); 41 final now = DateTime.now().toIso8601String(); 42 await db.transaction((txn) async { 43 await txn.delete('cached_posts'); 44 final batch = txn.batch(); 45 for (final post in posts) { 46 batch.insert('cached_posts', { 47 'id': post.id, 48 'payload': jsonEncode(post.toJson()), 49 'cached_at': now, 50 }); 51 } 52 await batch.commit(noResult: true); 53 }); 54 return posts; 55 } 56 } </pre>

	Penjelasan: Repository posts menerapkan pola cache-first: data cache lokal tersedia lebih dahulu, kemudian remote dapat diambil dan disimpan kembali sebagai cache.
6	Buat provider cache-first lib/providers/post_providers.dart.

```

lib > providers > post_providers.dart > ...
1  import 'dart:async';
2
3  import 'package:flutter_riverpod/flutter_riverpod.dart';
4
5  import '../data/remote/post.dart';
6  import '../data/repositories/post_repository.dart';
7  import '../data/sync.dart';
8  import 'offline_providers.dart';
9
10 final postRepositoryProvider =
11     Provider<PostRepository>((ref) => PostRepository());
12
13 final postsProvider =
14     AsyncNotifierProvider<PostsNotifier, List<Post>>(PostsNotifier.new);
15
16 class PostsNotifier extends AsyncNotifier<List<Post>> {
17     @override
18     Future<List<Post>> build() async {
19         final repo = ref.watch(postRepositoryProvider);
20         final offline = ref.watch(forceOfflineProvider);
21
22         // 1. Selalu baca cache lokal terlebih dahulu.
23         final cached = await repo.readCachedPosts();
24
25         if (offline) {
26             if (cached.isEmpty) {
27                 throw const OfflineException(
28                     'Offline dan belum ada cache. Buka halaman ini sekali saat online.',
29                 );
30             }
31             return cached;
32         }
33
34         // 2. Belum ada cache: terpaksa menunggu jaringan.
35         if (cached.isEmpty) return repo.fetchAndCache();
36
37         // 3. Ada cache: tampilkan segera, refresh di background.
38         unawaited(_refreshInBackground(repo));
39         return cached;
40     }
41
42     Future<void> _refreshInBackground(PostRepository repo) async {
43         try {
44             final fresh = await repo.fetchAndCache();
45             state = AsyncData(fresh);
46         } catch (_) {
47             // Jaringan gagal: tetap tampilkan cache, jangan ganti UI dengan error.
48         }
49     }
50
51     /// Dipanggil oleh pull-to-refresh. Mengembalikan true bila berhasil.
52     Future<bool> refresh() async {
53         if (ref.read(forceOfflineProvider)) return false;
54         try {
55             final fresh = await ref.read(postRepositoryProvider).fetchAndCache();
56             state = AsyncData(fresh);
57             return true;
58         } catch (_) {
59             return false; // data lama (cache) tetap ditampilkan
60         }
61     }
62 }

```

Penjelasan: Provider cache-first mengatur proses loading/data/error sehingga halaman dapat menampilkan cache tanpa harus menunggu jaringan.

Buat halaman posts lib/pages/posts_page.dart.

```

lib > pages > posts_page.dart > ...
1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3
4  import '../providers/offline_providers.dart';
5  import '../providers/post_providers.dart';
6
7  class PostsPage extends ConsumerWidget {
8    const PostsPage({super.key});
9
10   @override
11   Widget build(BuildContext context, WidgetRef ref) {
12     final postsAsync = ref.watch(postsProvider);
13     final offline = ref.watch(forceOfflineProvider);
14
15     return Scaffold(
16       appBar: AppBar(title: const Text('Posts (cache-first)'),
17       body: Column(
18         children: [
19           if (offline)
20             Container(
21               width: double.infinity,
22               color: Colors.orange.shade100,
23               padding: const EdgeInsets.all(12),
24               child: const Text(
25                 'Mode offline: menampilkan data dari cache lokal',
26                 style: TextStyle(color: Colors.black87),
27               ), // Text
28             ), // Container
29           Expanded(
30             child: postsAsync.when(
31               loading: () => const Center(child: CircularProgressIndicator()),
32               error: (e, _) => Center(
33                 child: Padding(
34                   padding: const EdgeInsets.all(24),
35                   child: Text('$e', textAlign: TextAlign.center),
36                 ), // Padding
37               ), // Center
38               data: (posts) => RefreshIndicator(
39                 onRefresh: () async {
40                   final ok = await ref.read(postsProvider.notifier).refresh();
41                   if (!ok && context.mounted) {
42                     ScaffoldMessenger.of(context).showSnackBar(
43                       const SnackBar(
44                         content: Text('Gagal refresh, menampilkan cache'),
45                       ), // SnackBar
46                     );
47                   }
48                 },
49                 child: ListView.builder(
50                   itemCount: posts.length,
51                   itemBuilder: (context, index) {
52                     final post = posts[index];
53                     return ListTile(
54                       leading: CircleAvatar(child: Text(post.id.toString())),
55                       title: Text(post.title, maxLines: 1),
56                       subtitle: Text(post.body, maxLines: 2),
57                     ); // ListTile
58                   },
59                 ), // ListView.builder
60               ), // RefreshIndicator

```

	Penjelasan: PostsPage menampilkan hasil remote/cache dalam bentuk daftar dan memberi label bahwa strategi yang digunakan adalah cache-first.
8	Tambahkan tombol Posts dan Sinkronkan di AppBar NotesPage. <pre> 9 import '../data/sync.dart'; 10 import 'posts_page.dart'; 11 52 IconButton(53 tooltip: 'Posts (cache-first)', 54 icon: const Icon(Icons.article_outlined), 55 onPressed: () => Navigator.of(context).push(56 MaterialPageRoute(builder: (_) => const PostsPage()), 57), 58), // IconButton 59 IconButton(60 tooltip: 'Sinkronkan', 61 icon: const Icon(Icons.sync), 62 onPressed: () async { 63 final messenger = ScaffoldMessenger.of(context); 64 try { 65 final count = await ref.read(noteActionsProvider).sync(); 66 messenger.showSnackBar(SnackBar(67 content: Text(count == 0 68 ? 'Semua catatan sudah tersinkron' 69 : '\$count catatan berhasil disinkronkan'), // Text 70)); // SnackBar 71 } on OfflineException catch (e) { 72 messenger.showSnackBar(SnackBar(content: Text(e.message))); 73 } 74 }, 75), // IconButton </pre> Penjelasan: AppBar NotesPage ditambah akses ke halaman Posts dan tombol Sinkronkan agar kedua alur dapat diuji langsung dari aplikasi.
9	Tambahkan saklar offline di SettingsPage. <pre> 4 import '../providers/prefs_providers.dart'; 5 import '../providers/offline_providers.dart'; 6 54 SwitchListTile(55 secondary: const Icon(Icons.wifi_off), 56 title: const Text('Paksa mode offline'), 57 subtitle: const Text('Simulasi offline untuk demo dan testing'), 58 value: offline, 59 onChanged: (_) => 60 ref.read(forceOfflineProvider.notifier).toggle(), 61), // SwitchListTile 62], 63), // ListView 64); // Scaffold 65 } 66 } 67 </pre> Penjelasan: SettingsPage menyediakan saklar Paksaan offline untuk pengujian sinkronisasi dan cache tanpa memerlukan pemutusan jaringan sebenarnya.
10	Hasil Run

	100 cache first
--	-----------------

← Posts (cache-first)

1

sunt aut facere repellat provident
quia et suscipit
suscipit recusandae consequuntur expedita et

2

qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea

3

ea molestias quasi exercitationem
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad

4

eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum

5

nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est

6

dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis provident
sequi

7

magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe

8

dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum
facilis quibusdam animi sint suscipit qui sint

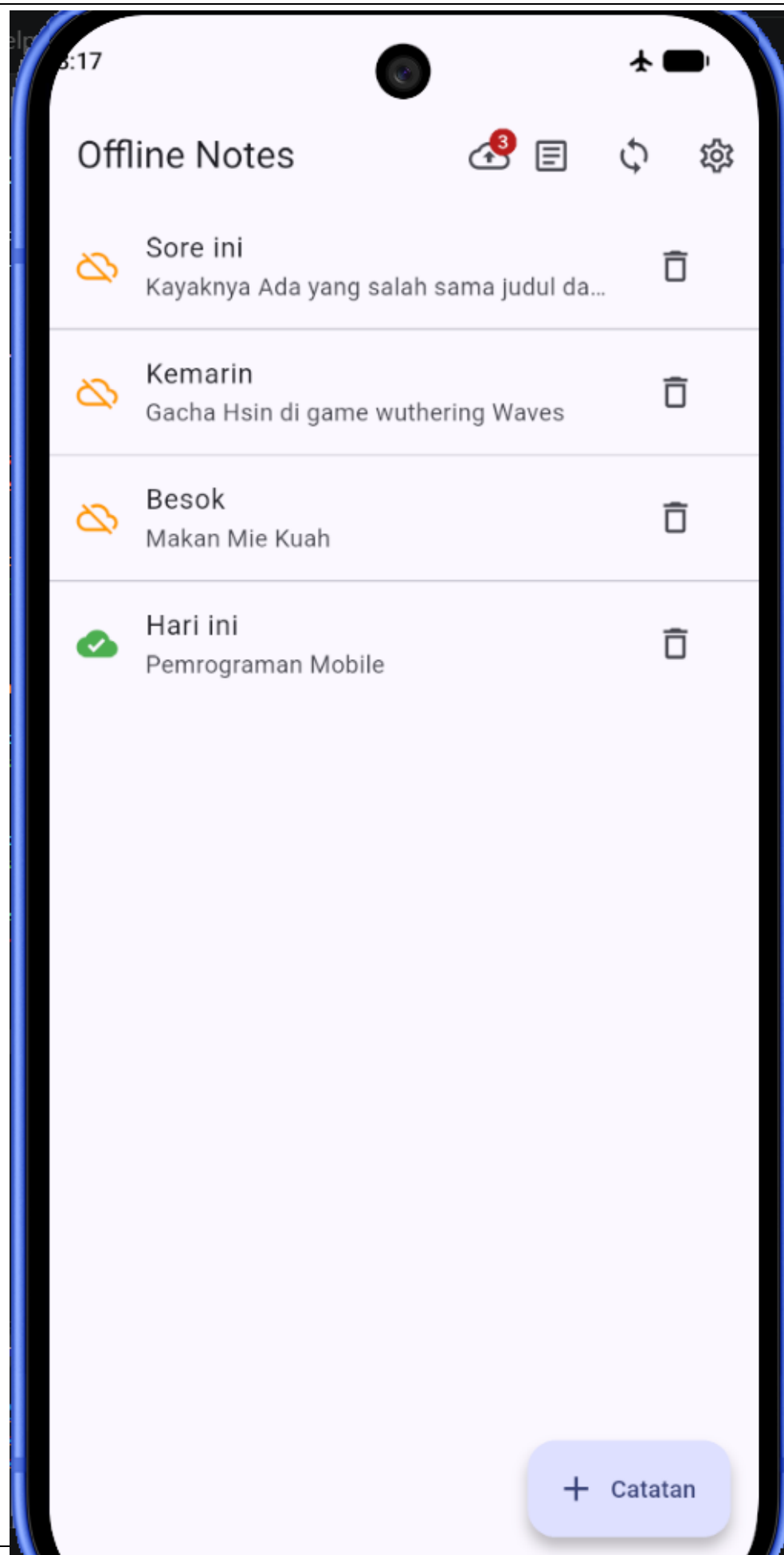
9

nesciunt iure omnis dolorem tempora et
consectetur animi nesciunt iure dolore
enim quia ad

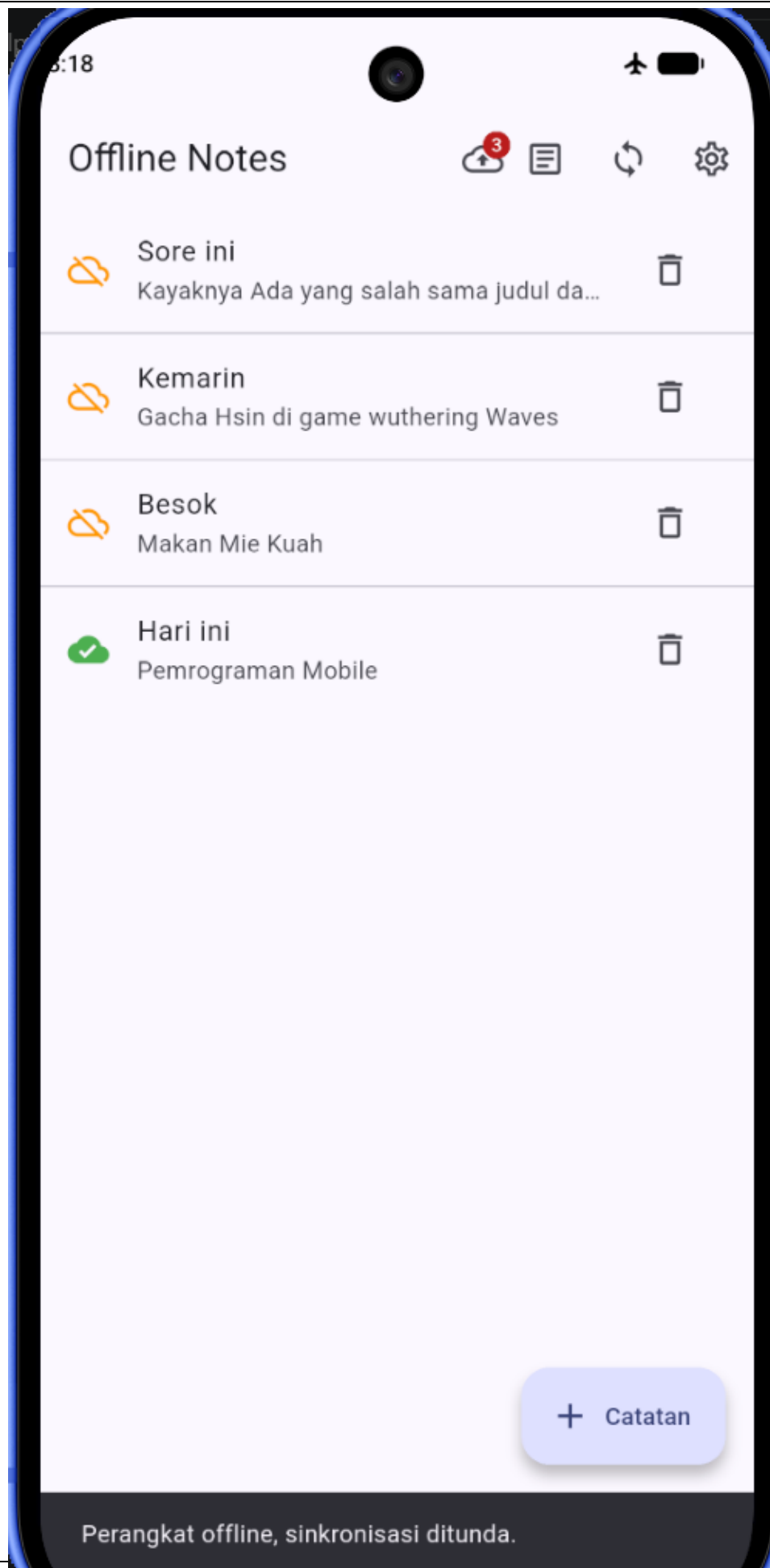
10

optio molestias id quia eum
quo et expedita modi cum officia vel magni
doloribus qui repudiandae

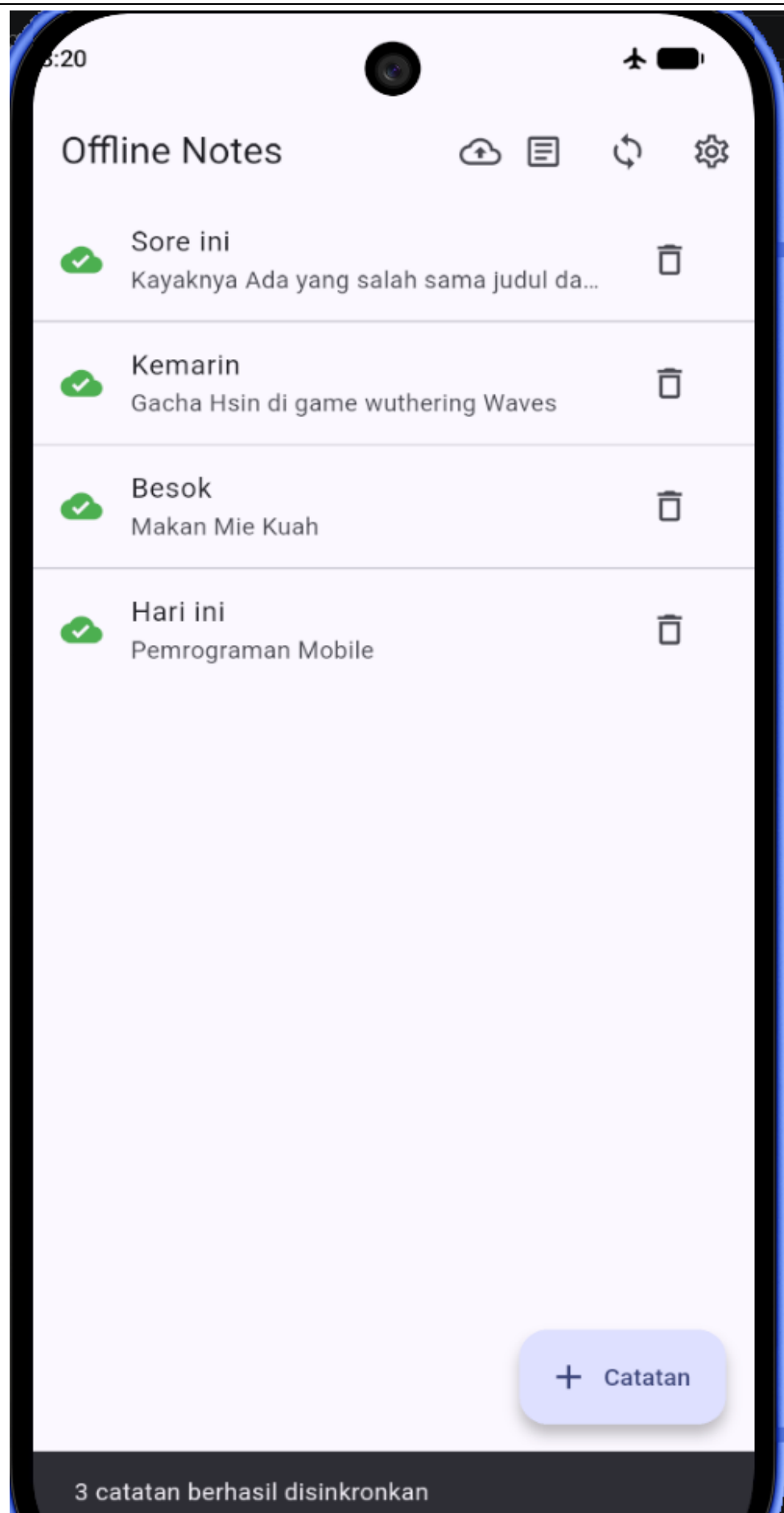
	Menambah 3 catatan
--	---------------------------



	Sinkronisasi 3 catatan dalam mode paksa offline
--	--



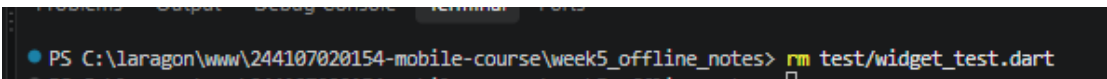
	Mode online
--	--------------------



	Penjelasan: Hasil run menunjukkan data posts dapat ditampilkan dengan cache-first dan skenario pengujian sinkronisasi/online dicatat pada bagian hasil. Screenshot yang tersedia terutama menunjukkan halaman Posts (cache-first).
11	Pertanyaan Praktikum 1. Apa perbedaan cache-first dan network-first? Berikan satu contoh data yang lebih cocok memakai network-first. Cache-first menggunakan data cache lokal terlebih dahulu agar UI cepat, lalu dapat memperbaruinya dari network. Network-first mencoba network terlebih dahulu dan memakai cache sebagai fallback. Contoh yang lebih cocok untuk network-first adalah harga saham atau harga produk real-time karena data terbaru lebih penting daripada kecepatan cache. 2. Jelaskan skenario kehilangan data yang dapat terjadi akibat markAllSynced(), lalu usulkan perbaikannya. Jika markAllSynced() dipanggil sebelum server benar-benar mengonfirmasi semua perubahan, dirty flag dapat berubah menjadi false sehingga catatan tidak lagi masuk antrean sinkronisasi walaupun sebenarnya belum terkirim. Perbaikannya adalah menandai tiap catatan synced hanya setelah upload item tersebut sukses; untuk sistem yang lebih kuat gunakan outbox/pending queue. 3. Mengapa diperlukan saklar forceOffline padahal sudah ada mode pesawat? forceOffline diperlukan karena mode pesawat bergantung pada kondisi perangkat/emulator dan tidak selalu nyaman atau deterministik untuk test otomatis. Saklar ini membuat skenario offline bisa direproduksi dengan langkah yang sama setiap kali. 4. Mengapa fetchAndCache() menulis cache di dalam transaksi? fetchAndCache() menulis cache dalam transaksi agar beberapa perubahan cache diperlakukan sebagai satu unit atomik. Jika salah satu penulisan gagal, seluruh transaksi dapat di-rollback sehingga cache tidak berhenti pada kondisi setengah diperbarui.

Skenario Uji	Hasil Aktual	Status (Lulus/Gagal)
Cache-first	Halaman Posts menampilkan daftar data pada tampilan "Posts (cache-first)".	Lulus
Menambah 3 catatan	Hasil run mencatat skenario penambahan 3 catatan pada pengujian sinkronisasi.	Lulus
Sinkronisasi 3 catatan saat forceOffline aktif	Skenario sinkronisasi offline dicatat dalam hasil run; forceOffline membuat kondisi tanpa jaringan dapat direproduksi.	Lulus
Sinkronisasi 3 catatan saat Online	Hasil run menyediakan skenario kembali ke mode online untuk melanjutkan sinkronisasi.	Lulus

Praktikum 5 – Pengujian dengan Repository Palsu

Langkah	Jawaban/Deskripsi
1	Hapus test bawaan dan buat test/note_test.dart. 

```

test > note_test.dart > ...
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2  import 'package:flutter_test/flutter_test.dart';
3  import 'package:week5_offline_notes/data/local/note.dart';
4  import 'package:week5_offline_notes/data/repositories/note_repository.dart';
5  import 'package:week5_offline_notes/data/sync.dart';
6  import 'package:week5_offline_notes/providers/note_providers.dart';
7
8  class FakeNoteRepository extends NoteRepository {
9      FakeNoteRepository({List<Note> items = const [], this.throwError = false})
10         : items = [...items],
11           super(openDb: () => throw UnimplementedError());
12
13     final List<Note> items;
14     final bool throwError;
15
16     @override
17     Future<List<Note>> fetchNotes() async {
18         if (throwError) throw Exception('db locked (simulasi)');
19         return items;
20     }
21
22     @override
23     Future<int> countDirty() async => items.where((n) => n.dirty).length;
24
25     @override
26     Future<void> markAllSynced() async {
27         for (var i = 0; i < items.length; i++) {
28             items[i] = items[i].copyWith(dirty: false);
29         }
30     }
31 }
32
33 Note _note(String title, {bool dirty = false, DateTime? at}) =>
34     Note(title: title, updatedAt: at ?? DateTime(2026, 9, 18), dirty: dirty);
35

```

```

Run | Debug
36 void main() {
Run | Debug
37 group('Model Note', () {
Run | Debug
38 test('fromMap aman terhadap field yang hilang', () {
39     final note = Note.fromMap({'title': 'Belanja'});
40     expect(note.title, 'Belanja');
41     expect(note.body, '');
42     expect(note.dirty, isFalse);
43 }); // test('fromMap aman terhadap field yang hilang')
44
Run | Debug
45 test('flag dirty bertahan pada serialisasi', () {
46     final note = _note('a', dirty: true);
47     final restored = Note.fromMap(note.toMap());
48     expect(restored.dirty, isTrue);
49     expect(restored.updatedAt, note.updatedAt);
50 }); // test('flag dirty bertahan pada serialisasi')
51 }); // group('Model Note')
52
Run | Debug
53 group('Provider dengan repository palsu', () {
Run | Debug
54 test('notesProvider sukses', () async {
55     final container = ProviderContainer(
56         overrides: [
57             noteRepositoryProvider.overrideWithValue(
58                 FakeNoteRepository(items: [_note('Tes')]),
59             ),
60         ],
61     ); // ProviderContainer
62     addTearDown(container.dispose);
63
64     final notes = await container.read(notesProvider.future);
65     expect(notes.length, 1);
66     expect(notes.first.title, 'Tes');
67 }); // test('notesProvider sukses')
68

```

```

69 test('notesProvider error', () async {
70     final container = ProviderContainer(
71         overrides: [
72             noteRepositoryProvider.overrideWithValue(
73                 FakeNoteRepository(throwError: true),
74             ),
75         ],
76     ); // ProviderContainer
77     addTearDown(container.dispose);
78
79     await expectLater(
80         container.read(notesProvider.future),
81         throwsA(isA<Exception>()),
82     );
83 }); // test('notesProvider error')
84 }); // group('Provider dengan repository palsu')
85
Run | Debug
86 group('Offline-first', () {
Run | Debug
87     test('syncNotes membersihkan semua catatan dirty', () async {
88         final repo = FakeNoteRepository(items: [
89             _note('a', dirty: true),
90             _note('b', dirty: true),
91             _note('c'),
92         ]); // FakeNoteRepository
93         final synced = await syncNotes(repo, latency: Duration.zero);
94         expect(synced, 2);
95         expect(await repo.countDirty(), 0);
96     }); // test('syncNotes membersihkan semua catatan dirty')
97

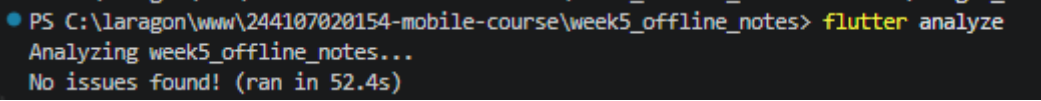
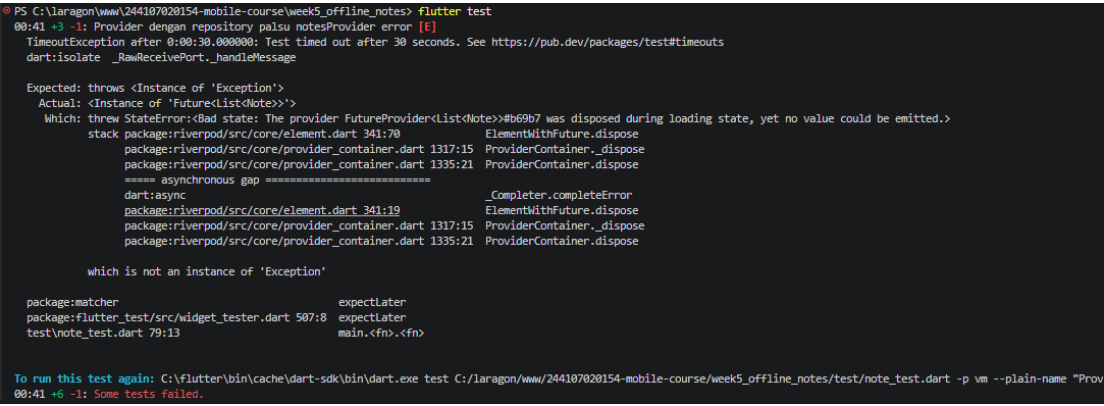
```

```

Run | Debug
98 test('syncNotes ditolak saat offline dan dirty tetap utuh', () async {
99     final repo = FakeNoteRepository(items: [_note('a', dirty: true)]);
100     await expectLater(
101         syncNotes(repo, offline: true),
102         throwsA(isA<OfflineException>()),
103     );
104     expect(await repo.countDirty(), 1);
105 }); // test('syncNotes ditolak saat offline dan dirty tetap utuh')
106
Run | Debug
107 test('resolveConflict memilih updatedAt terbaru', () {
108     final local = _note('lokal', at: DateTime(2026, 9, 18, 10));
109     final remote = _note('server', at: DateTime(2026, 9, 18, 11));
110     expect(resolveConflict(local, remote).title, 'server');
111     expect(resolveConflict(remote, local).title, 'server');
112 }); // test('resolveConflict memilih updatedAt terbaru')
113 }); // group('Offline-first')
114 }
115

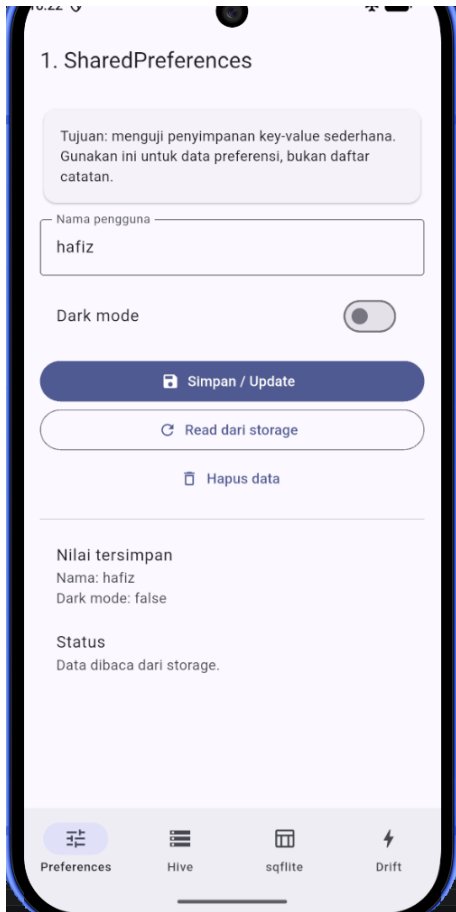
```

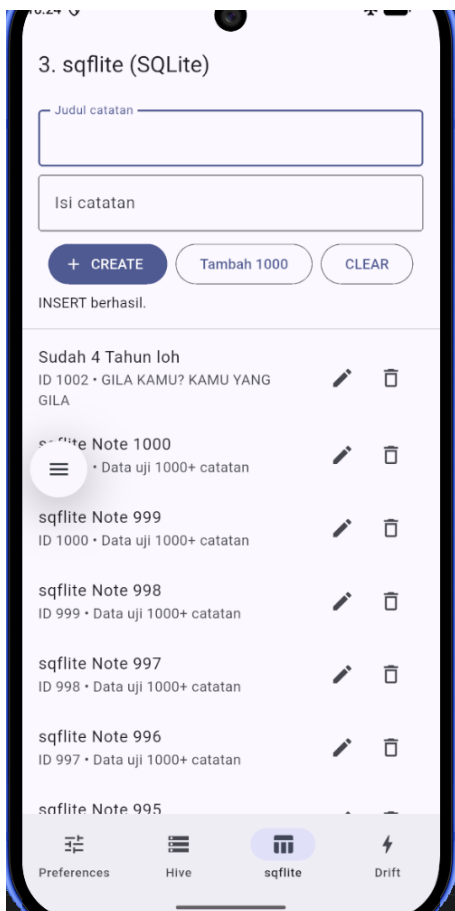
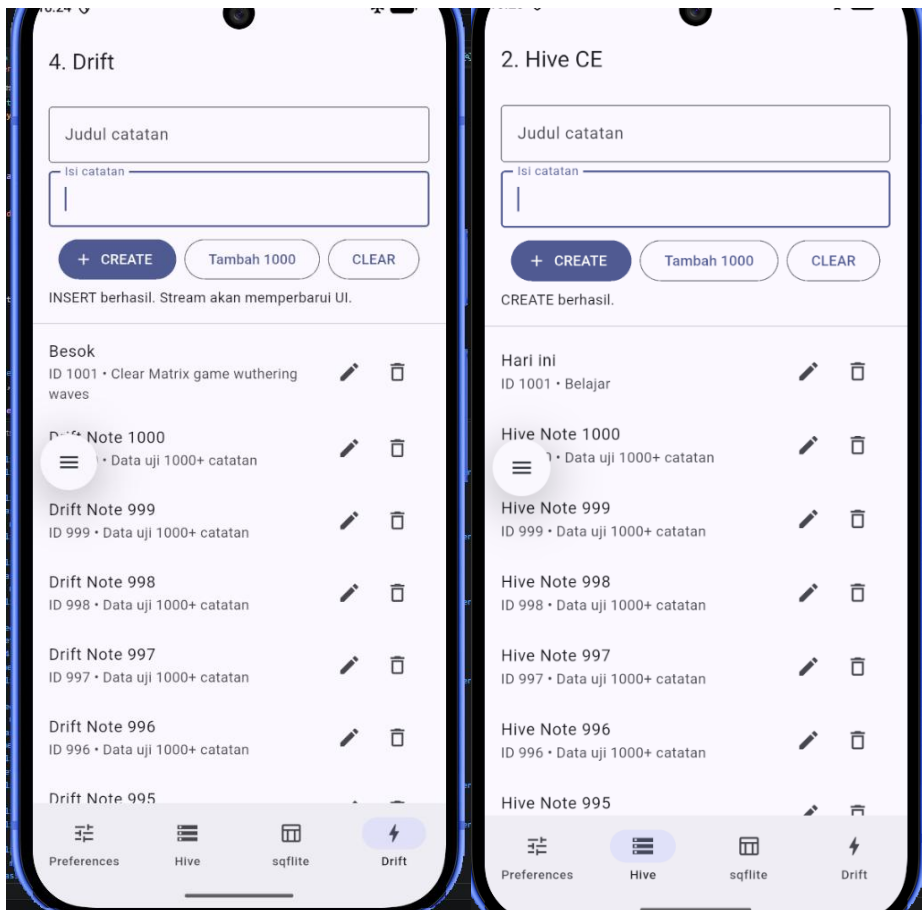
Penjelasan: FakeNoteRepository menggantikan database asli sehingga provider dan logika sinkronisasi dapat diuji dengan data yang dikontrol oleh test. Pendekatan ini mempercepat pengujian dan memudahkan simulasi error.

2	Jalankan analisis statis dan test.   Penjelasan: Analisis statis dan flutter test digunakan untuk menemukan masalah kode serta memastikan perilaku provider. Pada screenshot, pengujian aktual belum lulus karena test mencapai timeout dan terjadi mismatch state provider yang diharapkan.
3	Pertanyaan Praktikum 1. Mengapa kita menguji provider dengan ProviderContainer + overrideWithValue dan bukan dengan membuka database asli? ProviderContainer + overrideWithValue memungkinkan repository palsu disuntikkan ke provider sehingga test fokus pada logika provider tanpa membuka database asli. Hasilnya lebih cepat, terisolasi, dan deterministik serta tidak bergantung pada file database di perangkat. 2. Apa manfaat parameter latency pada syncNotes bagi pengujian? latency memungkinkan pengujian mensimulasikan waktu respons network/sync. Parameter ini berguna untuk memeriksa state loading, urutan operasi asynchronous, serta perilaku ketika sinkronisasi tidak selesai seketika. 3. Tuliskan satu test tambahan yang menurut Anda penting namun belum ada, beserta alasannya. Test tambahan yang penting adalah "sinkronisasi gagal mempertahankan dirty flag". Setelah repository dibuat melempar error pada upload/sync, assert bahwa catatan tetap dirty. Test ini penting agar kegagalan network tidak menyebabkan data dianggap sudah tersinkron padahal belum.

AI Prompt Challenge

Gunakan AI coding assistant (Cursor, Copilot, Claude Code, atau tool setara) dengan prompt berikut:





Ringkasan Output AI dan Verifikasi

Hasil perbandingan menunjukkan bahwa SharedPreferences paling tepat untuk preferensi sederhana seperti tema; untuk catatan skala 1000+ dibutuhkan storage yang mendukung query, indeks, dan struktur data yang lebih baik. sqflite cocok untuk implementasi saat ini karena proyek sudah menggunakan SQLite dan kebutuhan CRUD dapat dilayani dengan SQL, sedangkan Drift memberi type-safety dan reaktivitas yang lebih kuat dengan tambahan setup code generation.

Trade-off yang ditemukan: SharedPreferences memiliki boilerplate paling kecil tetapi tidak cocok untuk koleksi besar; Hive lebih mudah untuk object/key-value dan dapat reaktif, tetapi relasi/query kompleks harus dikelola aplikasi; sqflite memberi kontrol SQL dan relasi tetapi stream/type-safety tidak built-in; Drift memiliki boilerplate/setup lebih besar namun unggul untuk query type-safe dan stream/watch.

AI Verification Checklist

Sebelum rekomendasi AI diterima, verifikasi dan catat temuan Anda di README:

- Apakah AI menempatkan daftar catatan di SharedPreferences? (Tolak: rapuh untuk koleksi.)
Temuan: Tidak; daftar catatan seharusnya disimpan di SQLite/Hive/Drift, bukan SharedPreferences.
- Apakah skema AI mendukung antrean sync (dirty flag / updated_at) atau hanya CRUD polos?
Temuan: Ya, rancangan memakai dirty flag/updated_at sehingga perubahan dapat dilacak sebelum sync.
- Apakah klaim “real-time” AI didukung stream (Drift watch) atau hanya asumsi?
Temuan: Reaktivitas real-time harus didukung stream/watch; sqflite sendiri tidak menyediakan watch otomatis seperti Drift.
- Apakah estimasi boilerplate AI masuk akal setelah Anda mencoba instalasinya (flutter pub add + migrasi skema)?
Temuan: Secara struktur masuk akal; implementasi yang ada menunjukkan sqflite membutuhkan repository dan mapping model, sedangkan Drift memerlukan setup generator tambahan.
- Keputusan final Anda beserta alasannya — boleh berbeda dari rekomendasi AI selama berargumen.

SharedPreferences untuk preferensi; sqflite untuk catatan pada proyek ini. Drift menjadi opsi peningkatan bila kebutuhan stream dan type-safety bertambah.

Lembar Kerja: Tabel Perbandingan Storage

Salin tabel berikut ke docs/perbandingan-storage.md, isi berdasarkan hasil verifikasi Anda

Kriteria	SharedPreferences	Hive	sqlite	Drift
Kompleksitas query	Key-value sederhana.	Box/key-value; query terbatas.	SQL penuh: SELECT/WHERE/JOIN/INDEX	SQL + generated query API type-safe.
Dukungan relasi	Tidak mendukung relasi.	Relasi dikelola di level aplikasi.	Mendukung relasi/foreign key SQLite.	Mendukung relasi dan join.
Reaktivitas (stream)	Tidak ada stream query bawaan.	Ada listener/watch pada box.	Tidak built-in; perlu mekanisme stream/callback.	Sangat baik melalui watch/stream query.
Type-safety	Tipe dasar terbatas.	Bisa dengan adapter; setup model diperlukan.	Mapping manual; type-safety dijaga di kode.	Tinggi berkat generated classes/query.
Ukuran boilerplate	Sangat kecil.	Kecil–sedang.	Sedang.	Sedang–tinggi karena code generation.
Kemudahan testing	Mudah untuk layer repository/service.	Cukup mudah dengan box terisolasi.	Baik dengan test/temp DB atau fake repository.	Baik; database/query dapat diuji terisolasi.
Cocok untuk preferensi?	Ya, pilihan utama untuk preferensi.	Bisa, tetapi berlebihan untuk preferensi sederhana.	Bisa, tetapi terlalu berat untuk satu-dua pengaturan.	Bisa, tetapi overkill untuk preferensi sederhana.
Cocok untuk 1000+ catatan?	Tidak, tidak cocok untuk koleksi besar/terstruktur.	Ya, cocok untuk data object lokal sederhana.	Ya, cocok untuk 1000+ catatan dengan SQL/index.	Ya – sangat cocok untuk 1000+ dengan query type-safe/stream.

Kriteria	SharedPreferences	Hive	sqlite	Drift
Keputusan & alasan	Pilih untuk tema/preferensi karena sederhana dan ringan.	Alternatif bila fokus pada key-value object lokal dan ingin API yang praktis	Dipilih untuk proyek saat ini karena sudah sesuai CRUD SQLite, query, transaksi, dan data 1000+.	Pilihan upgrade bila aplikasi membutuhkan stream/watch dan type-safety lebih kuat.

Pertanyaan Refleksi

1. Mengapa daftar catatan tidak boleh disimpan di SharedPreferences? Apa yang rusak jika aturan ini dilanggar?

SharedPreferences dibuat untuk data preferensi kecil berupa key-value, bukan koleksi catatan. Jika daftar catatan dipaksa masuk ke SharedPreferences, data harus diserialisasi menjadi satu nilai besar, pencarian/edit per item menjadi tidak efisien, struktur dan integritas data lebih rapuh, serta sulit mendukung query, relasi, dan dirty flag secara rapi.

2. Kapan cache-first cukup, dan kapan Anda membutuhkan strategi lain (misalnya network-first untuk data harga real-time)?

Cache-first cukup ketika aplikasi membutuhkan respons cepat dan data masih boleh sedikit tertinggal, misalnya daftar konten yang tidak berubah setiap detik. Strategi seperti network-first lebih tepat untuk data yang sangat sensitif terhadap kebaruan, misalnya harga real-time, saldo rekening, atau stok yang cepat berubah.

3. Bagaimana dirty flag berubah menjadi antrean sync tanpa memblokir UI? Kapan antrean terpisah (tabel outbox) menjadi perlu?

Dirty flag dapat dianggap sebagai antrean sederhana: setiap create/update/delete membuat dirty=1, lalu worker/service mengambil item dirty dan memprosesnya secara asynchronous. Setelah server mengonfirmasi sukses, item diubah menjadi dirty=0. UI tetap responsif karena sync berjalan di luar build. Tabel outbox menjadi perlu ketika operasi sync makin kompleks, perlu retry, urutan operasi, idempotensi, atau audit perubahan.

4. Bagian mana dari rekomendasi AI yang Anda tolak, dan mengapa?

Rekomendasi yang saya tolak adalah anggapan bahwa SharedPreferences cocok untuk menyimpan daftar catatan 1000+ atau bahwa sqflite otomatis real-time tanpa stream tambahan. Untuk kebutuhan koleksi dan query, SQLite/Drift lebih tepat; untuk reaktivitas, harus ada stream/watch atau mekanisme invalidasi yang jelas.